

Распределённая система телеметрии и управления шаговым двигателем

Крогаль К. Д., студент группы ИТП-31





Актуальность и цель исследования

Актуальность

Рост IoT и робототехники требует надёжных, масштабируемых систем удалённого мониторинга и управления актуаторами при строгих требованиях по задержке и надёжности.

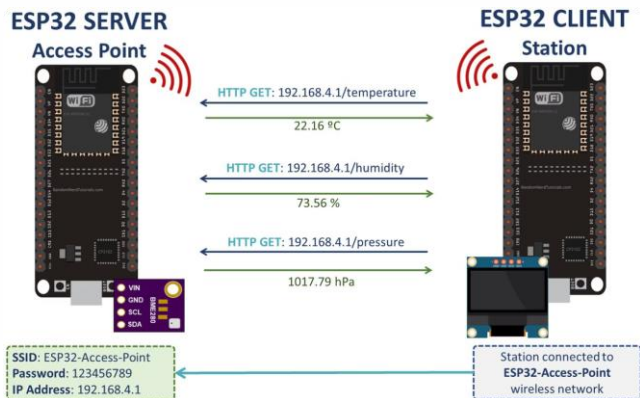
Проблема

Синхронные протоколы неэффективны для высокочастотных потоков телеметрии; необходим асинхронный подход с гарантированной доставкой и мониторингом сессий.

Цель и задачи

Проектирование и реализация распределённой системы сбора данных и управления шаговым двигателем: выбор аппаратной платформы, прошивки, серверной части и проведение тестирования.

Архитектура системы



Модель: периферийный узел (ESP32) → MQTT-брокер → серверный обработчик (Rust). Паттерн Publish/Subscribe обеспечивает асинхронный обмен и слабую связность.

- Протокол: MQTT с QoS 1, Last Will, heartbeat для мониторинга сессий
- Свойства: горизонтальное масштабирование, отказоустойчивость, маршрутизация задержки <100 мс



Аппаратная платформа

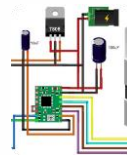


ESP32

Двухъядерный MCU с Wi-Fi, аппаратными таймерами и низкоуровневым управлением GPIO — основа периферийного узла.

HC-SR04

Ультразвуковой датчик 2–400 см, типичная точность ± 0.3 см; измерение по эхо-сигналу.



NEMA 17 + A4988

Шаговый двигатель с микрошагом 1/16; интерфейс STEP/DIR, требования по питанию и защите 5V→3.3V для ESP32.



Программный стек и инструменты

Сервер: Rust + Tokio, встроенный брокер rumqttd, клиент rumqtts. Конфигурация в TOML: лимиты соединений, payload-фильтры, политики ретеншна.

Прошивка: C/C++ (ESP-IDF/Arduino), JSON-упаковка (ArduinoJson), неблокирующий цикл опроса и FSM для управления соединением.



Инструменты

- VS Code
- Cargo / Rust toolchain
- Git, CI простого тестирования

Проект

Модульная структура с слоями: hw-abstraction, comms, control, server-api.

```
main.rs - ws (Workspace) - Visual Studio Code
main.rs x
low(unused)]
ain() {
// Hello
println!("Hello, world!");

// Numbers
let mut i = 5;
i += 3;
let f: f32 = 42.0;

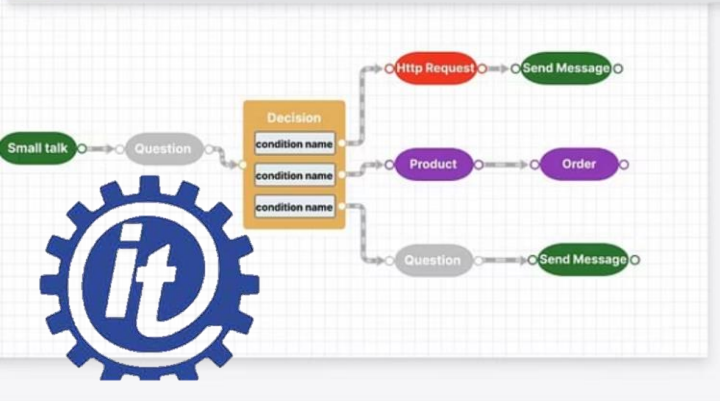
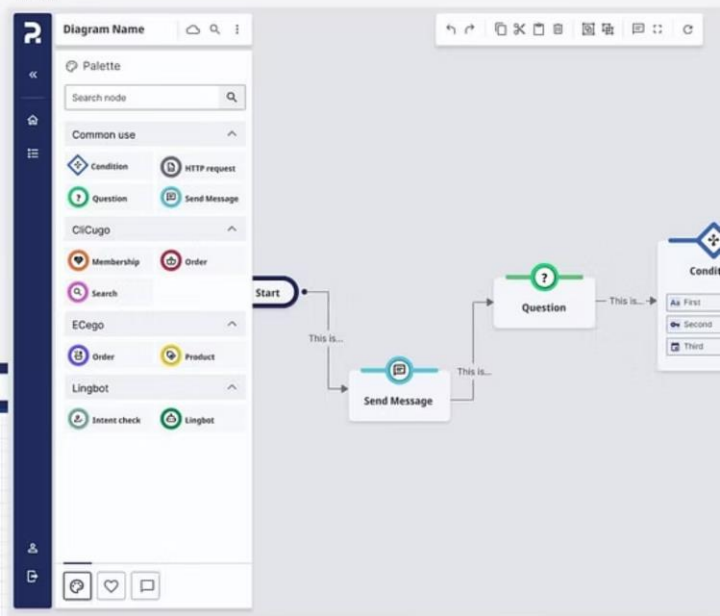
// Strings
let s = "SomeString";
let t = "SomeOtherString";
let mut u: String = "The".to_string();
u.push_str("ThirdString");

// Vec; works pretty well!
let nums = vec![1, 2, 3, 4, 5];

// HashMap; does not work well :(
let mut map = HashMap::<String, String>::new();
map.insert("some_key".to_string(), "some_value".to_string());
map.insert("some_other_key".to_string(), "some_other_val

// Stepping in the random crate
let x: u8 = random();
let y = random::<f64>();

// Profile (C++)
microprofile::init();
```



Ключевые алгоритмы

01

1. Измерение расстояния

Формирование 10 μ s триггера, захват длительности эхо с таймаутом; расчёт расстояния по скорости звука и температурной коррекции при необходимости.

02

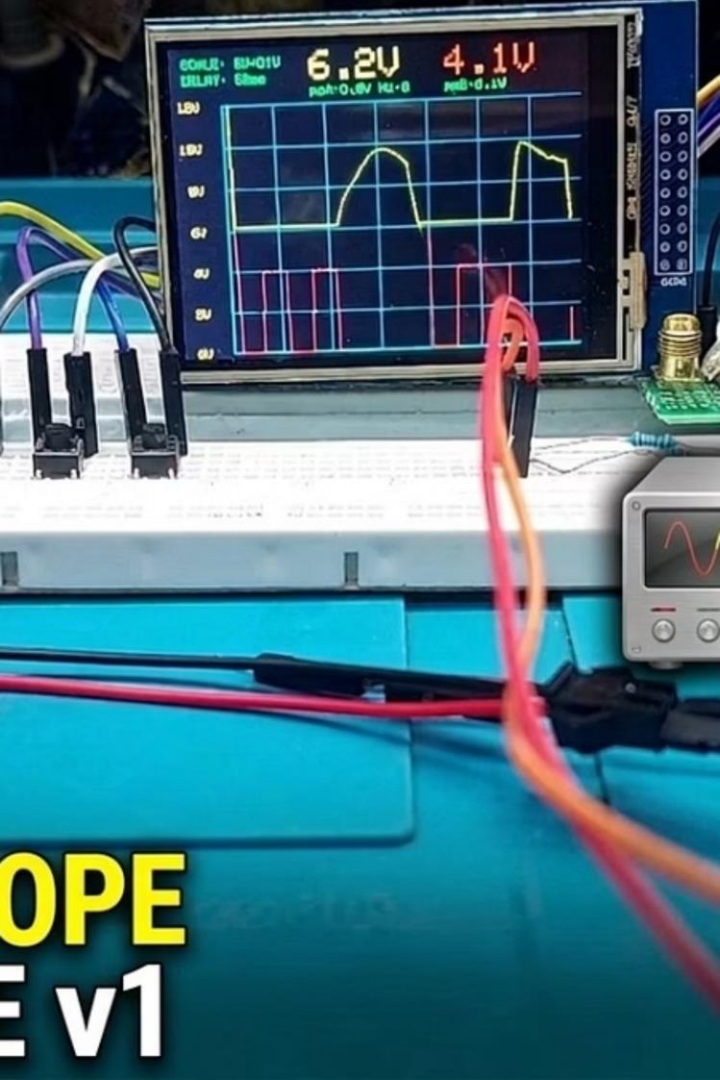
2. Управление сетью

Конечный автомат сессий: подключение → подписка → heartbeat → автоматический реконнект и экспоненциальный бек-оф при потере связи.

03

3. Серверная обработка

Десериализация JSON → валидация диапазона → пороговый анализ (<10 см) → логирование и публикация команд в управляющий топик.



Тестирование и верификация

Верификация включала: проверку электрических соединений, устойчивости сессии при искусственных разрывах сети, нагрузочные испытания брокера при множественных подписчиках.

- Апробация датчика: 50 замеров на контрольных расстояниях 20–300 см
- Стабильность доставки сообщений $\geq 99\%$

$\leq 0.5\%$

Относительная погрешность

Соответствие паспортной точности HC-SR04 в эксперименте

$\geq 99\%$

Доставка сообщений

Надёжность передачи при QoS 1 и повторных попытках

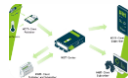


Практическая значимость



Применения

Мобильные роботы, автоматизированные склады, учебные лаборатории — архитектура легко адаптируется под реальные задачи.



Модульность

Добавление новых датчиков/актуаторов через подписку на топики без модификации ядра системы.



Ресурсоэффективность и безопасность

Минимальное использование RAM в прошивке; Rust-сервер исключает гонки данных и утечки памяти.



Перспективы развития

- Интеграция GUI (egui/iced) для визуализации телеметрии в реальном времени и ручного управления
- Сохранение истории в СУБД (PostgreSQL / MongoDB) для анализа и тренировки моделей
- Алгоритмы фильтрации: медианный фильтр, температурная компенсация, прогнозирование траектории
- Усиление безопасности: TLS, аутентификация клиентов, расширенный аудит логов



Заключение

Спроектирована и реализована полнофункциональная распределённая система телеметрии: стабильный сбор данных, передача и пороговая обработка в реальном времени. Решение масштабируемо, документировано и готово к опытной эксплуатации.

📄 Спасибо за внимание! Готов ответить на вопросы комиссии.

